

Shell

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

Informazioni generali

- **Shell**: interfaccia (modalità testo) tra utente al terminale ed il SO*:
 - l'interfaccia grafica tramite icone, finestre, etc... si chiama *graphical user interface* (GUI);
- Linux offre diverse shell: sh, csh, ksh, **bash**, zsh, ...:
 - **bash** (Bourne Again Shell, 1989): estensione della shell **sh** (del 1977, scritta da Steve Bourne).

*Riferimenti:

Cap. 1 par. 1.5.6; **Cap. 10** par. 10.2.3 e 10.2.4;

Alcune funzionalità

- ❑ Interpretazione linea di comando;
- ❑ Avvio dei programmi;
- ❑ Redirezione input-output e pipeline;
- ❑ Controllo dell'ambiente;
- ❑ Programmazione di shell;
- ❑ ...

Prompt e comandi

- ❑ La shell mostra un **prompt** (tipicamente simbolo **\$**);
- ❑ Quando l'utente digita una linea di comando, la shell ne estrae la prima parola interpretandola come un **programma** da eseguire*.

```
studente@opsys:~$ date
mar 17 ott 2023, 11:21:50, CEST
studente@opsys:~$ pluto
pluto: comando non trovato
```

* La shell crea un processo figlio, ed esegue il programma come figlio; la shell rimane in attesa che esso termini. Il simbolo **&**, aggiunto dopo il comando (per es., **date &**), consente alla shell di riprendere subito la propria esecuzione senza aspettare che il processo termini (esecuzione in **background**).

Una shell semplificata

```
while (TRUE) {                                /* ripeti per sempre */
    type_prompt( );                            /* mostra prompt sullo schermo */
    read_command(command, params);             /* legge riga di input dalla tastiera */

    pid = fork( );                             /* fork di un processo figlio */
    if (pid < 0) {
        printf("Unable to fork0);             /* condizione di errore */
        continue;                             /* ripeti il ciclo */
    }

    if (pid != 0) {
        waitpid (-1, &status, 0);             /* il genitore attende il figlio */
    } else {
        execve(command, params, 0);           /* il figlio svolge il lavoro */
    }
}
```

Flag, opzioni e parametri

- I comandi possono prevedere:
 - **flag** o **opzioni** che "modificano" il comportamento del programma e sono *tipicamente* identificati dal segno - ;
 - veri e propri **parametri** di ingresso.

```
studente@opsys:~$ date
mar 17 ott 2023, 11:27:13, CEST
studente@opsys:~$ date -u
mar 17 ott 2023, 09:27:17, UTC
studente@opsys:~$ date +%s
1697534844
```

Alcuni comandi relativi a file e directory

ls	List files in the directory		
ls -a	List files, include hidden files		
pwd	Show current directory	touch [file]	Create a new file
mkdir [name]	Create a directory	more [file]	Show file contents
rm [file]	Remove a file	head [file]	Show first 10 lines of a file
rm -r [directory]	Recursively remove directory	tail [file]	Show last 10 lines of a file
rm -rf [directory]	Force remove directory		
cp [file1] [file2]	Copy file1 to file2		
cp -r [directory1] [directory2]	Copy directory1 to directory2		
mv [filename1] [filename2]	Rename a file (rinomina o sposta un file)		

Comando **ps** (process status)

- ❑ Riporta lo stato dei processi (e thread) sul sistema;
- ❑ Per ogni processo riporta PID, stato, terminale controllante, tempo di CPU usato, comando corrispondente al processo;
- ❑ Esempi di invocazione:
 - **ps -ef** (e = tutti i processi; f = full format);
 - **ps -efL** (L = mostra i thread).

Help sui comandi (--help e man)

```
studente@opsys:~$ ls --help
```

```
Uso: ls [OPZIONE]... [FILE]...
```

```
List information about the FILES (the current directory  
by default).
```

```
[...]
```

```
studente@opsys:~$ man ls
```

```
LS(1) User Commands
LS(1)
NAME
  ls - list directory contents
SYNOPSIS
  ls [OPTION]... [FILE]...
DESCRIPTION
  List information about the FILES (the current directory by default). Sort entries alphabetically if none of -eftuvSUX nor --sort is specified.
  Mandatory arguments to long options are mandatory for short options too.
  -a, --all
      do not ignore entries starting with .
  -A, --almost-all
      do not list implied . and ..
```

Variabili d'ambiente

- Usate dalla shell (e processi figli). **Esempi:**

```
studente@opsys:~$ echo $PATH
```

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
```

```
studente@opsys:~$ echo $HOSTNAME
```

```
opsys
```

```
studente@opsys:~$ echo $SHELL
```

```
/bin/bash
```

```
studente@opsys:~$ echo $HOME
```

```
/home/studente
```

```
studente@opsys:~$ echo $USER
```

```
studente
```

```
studente@opsys:~$
```

Path (percorsi) assoluti e relativi

- ❑ Percorso **assoluto**: indica come giungere al file/directory partendo dalla directory radice (**root**);
- ❑ Percorso **relativo**: fa riferimento alla directory corrente.

```
studente@opsys:~$ pwd
/home/studente
studente@opsys:~$ mkdir mytest
studente@opsys:~$ touch mytest/myfile.txt
studente@opsys:~$ ls /home/studente/mytest/myfile.txt
/home/studente/mytest/myfile.txt
studente@opsys:~$ ls mytest/myfile.txt
mytest/myfile.txt
studente@opsys:~$ ls myfile.txt
ls: impossibile accedere a 'myfile.txt': File o directory
non esistente
```

Tipi di file

- ❑ **File ordinari**: sono sequenze di byte (byte stream) che possono contenere informazioni qualsiasi (dati, programmi sorgente, programmi oggetto):
 - il sistema non impone una particolare struttura al contenuto;
- ❑ **Directory**: sono contenitori di file (ordinari o speciali) e/o altre directory;
- ❑ **File speciali**: sono usati per astrarre i dispositivi di I/O (file & device independence):
 - i dispositivi sono 'integrati' nel file system (di solito in **/dev**)

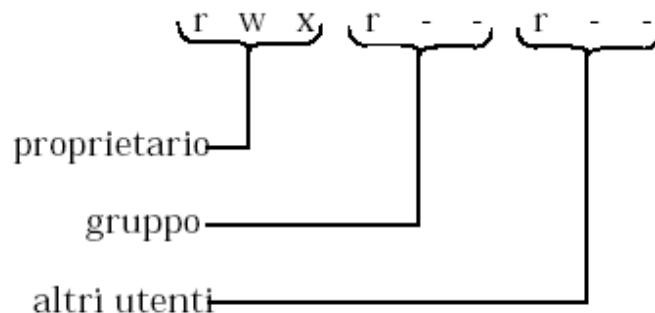
Permessi

- A file/directory possono essere attribuiti i seguenti permessi:

r : readable
w : writable
x : executable

} per { proprietario
gruppo
altri utenti

- Esempio:



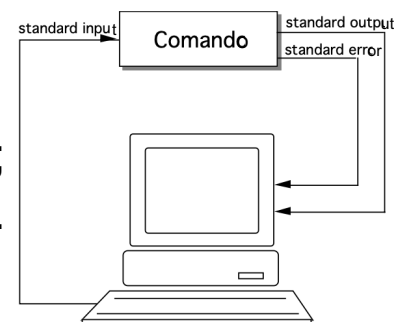
in binario: 111 100 100
in ottale: 7 4 4

Redirezione dell'I/O

- I programmi eseguiti da una shell hanno accesso automaticamente a **tre file "speciali"** (o stream):
 - standard **input** (per la lettura);
 - standard **output** (per la scrittura);
 - standard **error** (per la scrittura di errori).

- **In generale (default):**

- standard input → **tastiera;**
- standard output → **terminale utente;**
- standard error → **terminale utente.**



Redirezione dell'I/O

- È possibile associare dei file allo standard input e/o allo standard output e/o allo standard error*;
 - comando `< input_file`
 - comando `> output_file`
 - comando `2> error_file`

I caratteri in `input_file` sono letti dal programma come se provenissero dall'utente; l'output viene scritto sui file `output_file` o `error_file` invece di essere mostrato a video

***NOTA importante:** `<`, `>` e `2>` **sovrascrivono**; `<<`, `>>` e `2>>` fanno **append**.

Pipeline

- I comandi possono essere **concatenati** tramite una **pipe** (ossia un *tubo*);
- In BASH, l'**operatore |** implementa il meccanismo delle pipe nella forma seguente:

COMANDO₁ | COMANDO₂ | . . . | COMANDO_N

- **COMANDO_{i+1}** legge, **in input**, l'output di **COMANDO_i**

```
studente@opsys:~/mytest$ echo -n "This is a test" > myfile.txt
studente@opsys:~/mytest$ cat myfile.txt | grep "test"
This is a test
```


Comandi `su` e `sudo`*

- Il comando `su nome_utente` consente di cambiare utente (previo inserimento password per `nome_utente`):
 - se `nome_utente` è omesso, si assume che si intenda il `superuser` (o `root`), ovvero l'amministratore del sistema.
- Il comando `sudo` fornisce all'utente privilegi limitatamente al comando specificato.

```
studente@opsys:~/mytest$ cat /etc/sudoers
cat: /etc/sudoers: Permessi negati
studente@opsys:~/mytest$ sudo cat /etc/sudoers
[sudo] password di studente:
```

*`su` (switch user); `sudo` (super user do); `sudo su` (apre un ambiente root, previo inserimento password utente).

Gestione dei programmi utente ("pacchetti")

- `apt-get` è un tool a linea di comando usato in alcune distribuzioni Linux (Debian, Ubuntu) per la gestione dei programmi utente ("pacchetti")

```
studente@opsys:~$ ifconfig
```

Comando «ifconfig» non trovato, ma può essere installato con:

```
sudo apt install net-tools
```

Gestione dei programmi utente ("pacchetti")

```
studente@opsys:~$ sudo apt-get update
[sudo] password di studente:
Trovato:1 http://it.archive.ubuntu.com/ubuntu focal InRelease
... //omesso
Lettura elenco dei pacchetti... Fatto
studente@opsys:~$ sudo apt install net-tools
Lettura elenco dei pacchetti... Fatto
Generazione albero delle dipendenze
... //omesso
Configurazione di net-tools (1.60+git20180626.aebd88e-1ubuntu1)...
Elaborazione dei trigger per man-db (2.9.1-1)...
studente@opsys:~$
```

// esempio pacchetto net-tools

IMPORTANTE:

installazione pacchetti gcc e vim:

```
sudo apt install gcc (oppure sudo apt-get install build-essential)
sudo apt install vim
```

Remote login e SSH

- ❑ È possibile collegarsi ad un sistema UNIX in locale (login locale) oppure **tramite rete**.
- ❑ **Secure SHell (SSH)**: protocollo che consente di stabilire una sessione remota con interfaccia a riga di comando.

```
% ssh studente@192.168.64.2
studente@192.168.64.2's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-86-generic x86_64)
... //omesso
Last login: Tue Oct 17 21:34:51 2023 from 192.168.64.1
studente@opsys:~$
```

NOTA importante: il comando **ssh** è stato eseguito tramite la shell della macchina **HOST**; l'indirizzo IP 192.168.64.2 è l'indirizzo virtuale della macchina **GUEST**.

Remote login e SSH

- È necessario "predisporre" il **lato server*** (ossia la VM guest nel nostro caso):

```
studente@opsys:~$ sudo apt-get install openssh-server
[sudo] password di studente:
... //omesso
studente@opsys:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV
... //omesso
$ sudo systemctl start ssh
$ sudo systemctl status ssh
```

OpenSSH è una famiglia di tool basati sul protocollo SSH per controllare remotamente un computer, o trasferire file.

* <https://www.cyberciti.biz/faq/ubuntu-linux-install-openssh-server/>